Методы работы со строковыми переменными

Свойство length

Свойство **length** позволяет узнать длину строки. Под длиной понимается количество символов в ней.

Синтаксис

```
строка.length
```

Пример

Пусть в переменной **str** хранится строка. Давайте найдем ее длину:

```
let str = 'abcde';
console.log(str.length);
```

Результат выполнения кода:

```
5
```

Метод toLowerCase

Метод **toLowerCase** производит преобразование строки в нижний регистр (из больших букв делает маленькие). При этом возвращается новая строка, а исходная строка не меняется.

Синтаксис

```
строка.toLowerCase()
```

Пример

Давайте преобразуем все большие буквы строки в маленькие:

```
let str = 'Язык JAVASCRIPT';
console.log(str.toLowerCase());
```

Результат выполнения кода:

```
'язык javascript'
```

Метод toUpperCase

Метод **toUpperCase** производит преобразование строки в верхний регистр (из маленьких букв делает большие). При этом возвращается новая строка, а исходная строка не меняется.

Синтаксис

```
строка.toUpperCase()
```

Пример

Давайте преобразуем все буквы в большие:

```
let str = 'Язык JavaScript';
console.log(str.toUpperCase());
```

Результат выполнения кода:

```
'ЯЗЫК JAVASCRIPT'
```

Метод split

Метод **split** осуществляет разбиение строки в массив по указанному разделителю.

Разделитель указывается первым необязательным параметром. Если он не задан - вернется вся строка. Если он задан как пустые кавычки '' - то каждый символ строки попадет в отдельный элемент массива.

Вторым необязательным параметром можно указать максимальное количество элементов в получившемся массиве (см. примеры).

Синтаксис

```
строка.split([разделитель], [максимальное количество элементов])
```

Пример

Пусть дана некоторая строка с дефисами. Давайте разобьем эту строку в массив по разделителю '-':

```
let str = 'html-css-javascript';
let arr = str.split('-');

console.log(arr);
```

Результат выполнения кода:

```
['html', 'css', 'javascript']
```

Пример

Давайте опять разобьем строку по разделителю '-', однако вторым параметром укажем максимальное количество элементов в получившемся массиве, например, 2.

В этом случае в массив запишется только 2 элемента:

```
let str = 'html-css-javascript';
let arr = str.split('-', 2);

console.log(arr);
```

Результат выполнения кода:

```
['html', 'css']
```

Пример

Давайте каждый символ строки запишем в отдельный элемент массива:

```
let str = 'abcde';
let arr = str.split('');

console.log(arr);
```

Результат выполнения кода:

```
['a', 'b', 'c', 'd', 'e']
```

Пример

Давайте первые 3 символа строки запишем в отдельный элемент массива:

```
let str = 'abcde';
let arr = str.split('', 3);

console.log(arr);
```

Результат выполнения кода:

```
['a', 'b', 'c']
```

Пример

Сейчас с помощью **split** разбивается строка с числами. Обратите внимание на то, что в результате получится массив строк ['1', '2', '3', '4', '5'], а не массив чисел [1, 2, 3, 4, 5]:

```
let str = '12345';
let arr = str.split('');

console.log(arr);
```

Результат выполнения кода:

```
['1', '2', '3', '4', '5']
```

Пример

Давайте перевернем символы строки в обратном порядке. Для этого разобьем строку в массив с помощью **split** по разделителю '' (это разместит каждый символ строки в отдельный элемент массива), перевернем этот массив с помощью reverse и затем сольем перевернутый массив обратно с помощью join:

```
let str    = '123456789';
let arr1   = str.split('');
let arr2   = arr1.reverse();
let result = arr2.join('');

console.log(result);
```

Результат выполнения кода:

```
'987654321'
```

Пример

Упростим решение предыдущей задачи - сольем все команды в цепочку:

```
let str = '123456789';

let result = str.split('').reverse().join('');
console.log(result);
```

Результат выполнения кода:

```
'987654321'
```

Пример

Дана строка с цифрами. Найдем сумму цифр из этой строки. Для этого разобьем строку в массив, а затем переберем этот массив и найдем его сумму. Нас ждет подвох: **split** возвращает строки, поэтому при суммировании преобразуем эти цифры-строки в настоящие числа с помощью **Number**:

```
let str = '12345';
let arr = str.split('');
let sum = 0;

for (let i = 0; i < arr.length; i++) {
      sum += Number(arr[i]);
}

console.log(sum);
```

Результат выполнения кода:

```
15
```

Пример

Дано число. Давайте запишем каждую цифру этого числа в отдельный элемент массива. Тут есть подвох - **split** применяется только к строкам, а у нас число. Преобразуем вначале число к строке с помощью **String**, а затем применим **split**:

```
let num = 12345;
let str = String(num);
let arr = str.split('');

console.log(arr);
```

Результат выполнения кода:

```
['1', '2', '3', '4', '5']
```

Метод substr

Метод **substr** возвращает подстроку из строки (исходная строка при этом не изменяется).

Первый параметр задает номер символа, с которого метод начинает отрезать (учтите, что нумерация с нуля), а второй параметр - сколько символов отрезать.

Первый параметр может принимать отрицательные значения. В этом случае отсчет символа, с которого начинается обрезание, будет идти с конца строки. Причем последний символ имеет номер **-1**, предпоследний - **-2** и так далее.

Второй параметр не является обязательным, если он не указан, то вырезаны будут все символы до конца строки.

Синтаксис

```
строка.substr(откуда отрезать, [сколько символов отрезать])
```

Пример

Пусть дана строка. Давайте вырежем из нее первые **3** символа:

```
let str = 'abcde';
let sub = str.substr(0, 3);

console.log(sub);
```

Результат выполнения кода:

```
'abc'
```

Пример

Давайте теперь вырежем символы с 2-ой позиции и до конца строки (для этого не зададим второй параметр метода):

```
let str = 'abcde';
let sub = str.substr(2);

console.log(sub);
```

Результат выполнения кода:

```
'cde'
```

Пример

Давайте вырежем подстроку с **3**-го символа с конца и возьмем **2** символа:

```
let str = 'abcde';
let sub = str.substr(-3, 2);

console.log(sub);
```

Результат выполнения кода:

```
'cd'
```

Пример

А теперь давайте вырежем подстроку с **3**-го символа с конца и возьмем всю оставшуюся строку до конца (для этого не зададим второй параметр метода):

```
let str = 'abcde';
let sub = str.substr(-3);
```

```
console.log(sub);
```
Результат выполнения кода:
```
'cde'
```

Пример

Давайте вырежем последний символ строки:

```
let str = 'abcde';
let sub = str.substr(-1);

console.log(sub);
```
Результат выполнения кода:
```
'e'
```

Метод substring

Метод **substring** возвращает подстроку из строки (исходная строка при этом не изменяется).

Первый параметр задает номер символа, с которого метод начинает отрезать (учтите, что нумерация с нуля), а второй параметр - номер символа, на котором следует закончить вырезание (*символ с этим номером не включается в вырезанную часть*).

Второй параметр не является обязательным, если он не указан, то вырезаны будут все символы до конца строки.

Если первый параметр больше второго, тогда substringing ведет себя, как будто они поменялись местами. Например, написать **str.substringing(3, 0)** все равно что написать **str.substringing(0, 3)**.

Если какой-то из параметров больше, чем длина строки - он считается равным длине строки.

Отрицательные значения приравниваются к нулю.

Синтаксис

```
строка.substring(откуда начать отрезать, [докуда отрезать])
```

Пример

Пусть дана строка. Давайте вырежем из нее символы с 1-го по 3-тий:

```
let str = 'abcde';
let sub = str.substring(1, 3);

console.log(sub);
```
Результат выполнения кода (*символ с номером 3 не включится в вырезанную часть*):
```
'bc'
```

Пример

Давайте теперь вырежем символы с 1-го и до конца строки (для этого не зададим второй параметр метода):

```
let str = 'abcde';
let sub = str.substring(1);

console.log(sub);
```

Результат выполнения кода:

```
'bcde'
```

Метод slice

Метод **slice** возвращает возвращает подстроку из строки (исходная строка при этом не изменяется).

Первым параметром указывается номер символа строки, с которого начинается вырезание, а вторым параметром - номер символа, на котором закончится вырезание (*при этом символ с этим номером не включится в вырезанную часть*).

Второй параметр не является обязательным. Если его не указать - подстрока возьмется с указанного в первом параметре символа до конца строки.

Второй параметр также может принимать отрицательные значения. В этом случае отсчет символа, на котором закончится обрезание, начинается с конца строки. Причем последний символ имеет номер **-1**, предпоследний - **-2** и так далее.

К примеру, если написать **slice(1, -1)**, то вырезанная часть возьмется с первого символа включительно по последний не включая его. Если вам нужно включить и последний символ - нужно просто не задавать второй параметр для **slice**, вот так: **slice(1)**.

Если второй параметр меньше первого - возвратится пустая строка.

Синтаксис

```
строка.slice(откуда отрезать, [докуда отрезать])
```

Пример

Пусть дана строка. Давайте вырежем из нее символы с 1-го по 3-тий:

```
let str = 'abcde';
let sub = str.slice(1, 3);

console.log(sub);
```

Результат выполнения кода (*символ с номером 3 не включится в вырезанную часть*):

```
'bc'
```

Пример

Давайте теперь вырежем символы с 1-го и до конца строки (для этого не зададим второй параметр метода):

```
let str = 'abcde';
let sub = str.slice(1);
```

```
console.log(sub);
```
Результат выполнения кода:
```
'bcde'
```

Пример

Давайте теперь вырежем символы с позиции **1** по позицию **-2**:

```
let str = 'abcde';
let sub = str.slice(1, -2);

console.log(sub);
```
Результат выполнения кода (*символ с номером -2 не включится в вырезанную часть*):
```
'bc'
```

Метод concat

Метод **concat** соединяет указанные строки. Количество строк для объединения не ограниченно.

По сути действие этого метода аналогично операции '+' для слияния строк.

Синтаксис

```
строка.concat(еще строка, и еще строка, и еще...)
```

Пример

Давайте с помощью метода **concat** сольем 3 строки в одну:

```
let str1 = 'string1';
let str2 = 'string2';
let str3 = 'string3';

let result = str1.concat(str2, str3);
console.log(result);
```
Результат выполнения кода:
```
'string1string2string3'
```

Пример

Можно применить метод к пустой строке. В этом случае все сливаемые строки передадутся параметрами метода:

```
let str1 = 'string1';
let str2 = 'string2';
let str3 = 'string3';

let result = ''.concat(str1, str2, str3);
```

```
console.log(result);
```
Результат выполнения кода:

```
'string1string2string3'
```

Метод indexOf

Метод **indexOf** осуществляет поиск подстроки (указывается первым параметром) в строке.

Метод вернет позицию первого совпадения, а если оно не найдено, то вернет **-1**.

Вторым параметром (он необязателен) можно передать номер символа, откуда следует начинать поиск.

Метод чувствителен к регистру символов.

Синтаксис

```
строка.indexOf(что ищем, [откуда начинать поиск]);
```

Пример

В данном примере в строке 'Я учу учу Javascript' ищется слово 'учу'. Метод вернет 2 (с этой позиции начинается первое слово 'учу' в строке):

```
let str = 'Я учу учу Javascript';
console.log(str.indexOf('учу'));
```
Результат выполнения кода:

```
2
```

Пример

В данном примере в строке 'Я учу учу Javascript' также ищется слово 'учу', однако поиск начинается с 5-той позиции (я указал это вторым параметром). Метод вернет **6** (с этой позиции начинается **второе** слово 'учу' в строке):

```
let str = 'Я учу учу Javascript';
console.log(str.indexOf('учу', 5));
```
Результат выполнения кода:

```
6
```

Пример

В данном примере метод вернет -1, так как подстроки 'PHP' в строке 'Я учу Javascript' нету:

```
let str = 'Я учу Javascript';
console.log(str.indexOf('PHP', 5));
```
Результат выполнения кода:

```
-1
```

Пример

В данном примере метод вернет -1, так как он чувствителен к регистру (Javascript не равно JaVaScrIpt):

```
let str = 'Я учу Javascript';
console.log(str.indexOf('JaVaScrIpt'));
```

Результат выполнения кода:

```
-1
```

Пример

В данном примере метод вернет -1, так как после выбранной позиции совпадения отсутствуют:

```
let str = 'Я учу Javascript';
console.log(str.indexOf('учу', 8));
```

Результат выполнения кода:

```
-1
```

Метод lastIndexOf

Метод **lastIndexOf** осуществляет поиск подстроки (указывается первым параметром) в строке. Поиск ведется с конца строки.

Метод вернет позицию первого совпадения, а если оно не найдено, то вернет **-1**.

Вторым параметром (он необязателен) можно передать номер символа, откуда следует начинать поиск.

Метод чувствителен к регистру символов.

Синтаксис

```
строка.lastIndexOf(что ищем, [откуда начинать поиск]);
```

Пример

В данном примере в строке 'Б..Б..Б' ищется буква 'Б'. Так как поиск начинается с конца строки, то метод вернет **6** (позиция последней буквы 'Б'):

```
let str = 'Б..Б..Б';
console.log(str.lastIndexOf('Б'));
```

Результат выполнения кода:

```
6
```

Пример

В данном примере с помощью второго параметра указано, что поиск следует начинать с пятой позиции (это предпоследний символ строки). Поиск все равно будет идти справа

налево, но уже начиная не с конца строки, а с 5-той позиции. Это значит, что найдется буква 'Б', которая стоит посередине, и метод выведет 3:

```
let str = 'Б..Б..Б';
console.log(str.lastIndexOf('Б', 5));
```

Результат выполнения кода:

```
3
```

Пример

В данном примере в строке 'Б..Б..Б' ищется буква 'б'. Метод выведет -1, так как он чувствителен к регистру символов (мы ищем маленькую букву 'б', а в строке есть только большие):

```
let str = 'Б..Б..Б';
console.log(str.lastIndexOf('б'));
```

Результат выполнения кода:

```
-1
```